

ALANYA Optimisations de la base de données

24 Août 2026

Optimisation 1 — Index sur les chemins de lecture/écriture les plus chauds

```
ALTER TABLE message ADD INDEX idx_message_conv_status
(conversationID, status, senderID);

ALTER TABLE message ADD INDEX idx_message_conv_msgid
(conversationID, msgID);

ALTER TABLE job_queue ADD INDEX idx_job_ready
(failed_at, locked_at, run_after, id);

ALTER TABLE users ADD INDEX idx_users_fcm_token (fcm_token(191));

ALTER TABLE users DROP INDEX idx_users_phone;
```

Commentaire. Les deux premiers index évitent un balayage complet de l'historique d'une conversation à chaque accusé de lecture et à chaque synchronisation delta. Le troisième supprime un balayage complet de la file de tâches à chaque cycle de traitement (jusqu'à 20 fois par seconde). Le dernier DROP retire un index strictement redondant avec la contrainte UNIQUE existante.

Optimisation 2 — Élargissement de statut.ID en BIGINT

```
ALTER TABLE statut_views DROP FOREIGN KEY fk_sv_statut;
ALTER TABLE statut_i18n DROP FOREIGN KEY fk_statut_i18n;
ALTER TABLE welcome_status_delivery DROP FOREIGN KEY fk_wsd_statut;

ALTER TABLE statut MODIFY ID BIGINT UNSIGNED NOT NULL AUTO_INCREMENT;
ALTER TABLE statut_views MODIFY statutID BIGINT UNSIGNED NOT NULL;
ALTER TABLE statut_i18n MODIFY statut_id BIGINT UNSIGNED NOT NULL;
ALTER TABLE welcome_status_delivery MODIFY statut_id BIGINT UNSIGNED NOT NULL;
ALTER TABLE broadcast MODIFY statut_id BIGINT UNSIGNED NULL;

ALTER TABLE statut_views ADD CONSTRAINT fk_sv_statut
FOREIGN KEY (statutID) REFERENCES statut(ID) ON DELETE CASCADE;
ALTER TABLE statut_i18n ADD CONSTRAINT fk_statut_i18n
FOREIGN KEY (statut_id) REFERENCES statut(ID) ON DELETE CASCADE;
ALTER TABLE welcome_status_delivery ADD CONSTRAINT fk_wsd_statut
FOREIGN KEY (statut_id) REFERENCES statut(ID) ON DELETE CASCADE;
```

Commentaire. statut.ID était en INT signé (plafond 2,1 milliards), saturable en une quinzaine de mois au rythme visé. Les trois clés étrangères référençantes doivent être déposées avant l'élargissement de la colonne référencée, puis recrées à l'identique.

Optimisation 3 — Élargissement des colonnes sur les tables qui grossiront rapidement

```
ALTER TABLE broadcast_delivery
MODIFY conversID BIGINT UNSIGNED NULL,
```

```

MODIFY msgID BIGINT UNSIGNED NULL;

ALTER TABLE welcome_delivery
  MODIFY conversID BIGINT UNSIGNED NOT NULL;

ALTER TABLE trip_watcher
  MODIFY msgID BIGINT UNSIGNED NULL,
  MODIFY conversID BIGINT UNSIGNED NULL;

ALTER TABLE conv_participants
  MODIFY pendingJoinMsgID BIGINT UNSIGNED NULL,
  MODIFY unreadCount INT UNSIGNED NOT NULL DEFAULT 0;

```

Commentaire. Ces colonnes référençaient des identifiants BIGINT (`conversation.conversID`, `message.msgID`) tout en étant elles-mêmes en INT ou en type désaccordé, avec un risque de troncature silencieuse au-delà du plafond. `unreadCount` était en SMALLINT signé (plafond 32 767), débordable sur un genre très actif.

Optimisation 4 — Colonne générée indexée pour les réactions

```

ALTER TABLE message
  ADD COLUMN has_reactions TINYINT GENERATED ALWAYS AS
    (CASE WHEN reactions IS NOT NULL AND JSON_LENGTH(reactions) > 0
      THEN 1 ELSE 0 END) STORED,
  ADD INDEX idx_message_conv_hasreactions (conversationID, has_reactions);

```

Commentaire. Remplace un filtre `JSON_LENGTH(reactions) > 0` non indexable, qui provoquait un balayage complet de l'historique d'une conversation à chaque ouverture d'écran. La colonne STORED se recalcule automatiquement à chaque écriture, sans changement côté code d'écriture. Bien que l'idéal serait une table dédiée pour les réactions.

Optimisation 5 — Extraction de `message.mediaThumb`

```

CREATE TABLE message_thumb (
  msgID BIGINT NOT NULL PRIMARY KEY,
  thumb MEDIUMBLOB NOT NULL,
  CONSTRAINT fk_message_thumb FOREIGN KEY (msgID)
    REFERENCES message(msgID) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO message_thumb (msgID, thumb)
SELECT msgID, FROM_BASE64(mediaThumb) FROM message
WHERE mediaThumb IS NOT NULL AND mediaThumb <> '';

-- En attente (periode d'observation) :
-- ALTER TABLE message DROP COLUMN mediaThumb;

```

Commentaire. La vignette base64 (mesurée à 36 Ko en moyenne) était chargée par tous les `SELECT m.*` sur la table la plus utilisée du schéma, utile ou non au chemin concerné. Stockage en binaire (MEDIUMBLOB) plutôt qu'en base64 : environ 25 % d'économie. Le contrat JSON envoyé au client ne change pas (recodage à la lecture via `TO_BASE64`). Le `DROP COLUMN` final reste volontairement en attente comme filet de rollback.

Optimisation 6 — Extraction de `is_online/last_seen` vers `user_presence`

```

CREATE TABLE user_presence (
  alanyaID INT NOT NULL PRIMARY KEY,
  is_online TINYINT NOT NULL DEFAULT 0,
  last_seen DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

```

```

    CONSTRAINT fk_user_presence FOREIGN KEY (alanyaID)
      REFERENCES users(alanyaID) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO user_presence (alanyaID, is_online, last_seen)
SELECT alanyaID, is_online, last_seen FROM users;

-- En attente (periode d'observation) :
-- ALTER TABLE users DROP COLUMN is_online, DROP COLUMN last_seen;

```

Commentaire. Ces deux colonnes, écrites à chaque connexion/déconnexion/bascule premier plan, entraient en contention directe avec le verrou exclusif posé par `materializeForUser` sur la ligne `users`. Les extraire supprime cette contention sans changer la stratégie de verrouillage. Bascule complète des lectures/écritures effectuée côté application (4 sites d'écriture, une quinzaine de requêtes de lecture); le `DROP COLUMN` final reste en attente.

Optimisation 7 — Extraction des colonnes de chiffrement vers `message_crypto`

(À utiliser plus tard pour la crypto)

```

CREATE TABLE message_crypto (
  msgID BIGINT NOT NULL PRIMARY KEY,
  ciphertext MEDIUMBLOB NULL,
  dr_nonce VARBINARY(16) NULL,
  dr_header TEXT NULL,
  archive_blob MEDIUMBLOB NULL,
  signal_message_type TINYINT NULL,
  CONSTRAINT fk_message_crypto FOREIGN KEY (msgID)
    REFERENCES message(msgID) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO message_crypto
(msgID, ciphertext, dr_nonce, dr_header, archive_blob, signal_message_type)
SELECT msgID, ciphertext, dr_nonce, dr_header, archive_blob, signal_message_type
FROM message
WHERE ciphertext IS NOT NULL OR dr_nonce IS NOT NULL OR dr_header IS NOT NULL
   OR archive_blob IS NOT NULL OR signal_message_type IS NOT NULL;

ALTER TABLE message
  DROP COLUMN ciphertext,
  DROP COLUMN dr_nonce,
  DROP COLUMN dr_header,
  DROP COLUMN archive_blob,
  DROP COLUMN signal_message_type;

```

Commentaire. Cinq colonnes issues d'une fonctionnalité de chiffrement bout-en-bout en pause. La nouvelle table sera utilisée lors de la crypto

Optimisation 8 — Reclustering de `conv_participants`

```

ALTER TABLE conv_participants
  DROP PRIMARY KEY,
  DROP INDEX uq_conv_user,
  ADD PRIMARY KEY (conversID, alanyaID),
  ADD UNIQUE KEY uq_cp_id (id);

```

Commentaire. InnoDB organise physiquement une table selon sa clé primaire. Avec un identifiant synthétique, les participants d'une même conversation étaient dispersés sur des pages disque différentes. La nouvelle clé primaire (`conversID`, `alanyaID`) les colocalise, au bénéfice du chemin le plus sollicité de cette table : le fan-out des messages à tous les participants d'une conversation. L'identifiant `id` est conservé en clé secondaire (toujours `AUTO_INCREMENT`).